

WEEDOUT

Remediation and product cleanup

StrictSeal rejection remediation, conservative automated reachability, Free-only cleanup, CLI distribution, security review, and release evidence.

STATUS	DELIVERED
RELEASE	CLI v0.3.1
VALIDATION	1,725 backend + 137 frontend tests

ENGINEERING EVIDENCE REPORT / 30 AUGUST 2026

Outcome: the remediation is implemented end-to-end, the checksummed CLI release is public, and the web application now contains the previously missing PowerShell installer route.

Date: 30 August 2026 Scope: StrictSeal rejection remediation, Free-only product cleanup, CLI distribution, landing page, admin shell, security review, migration, regression coverage, and release validation.

Executive result

The requested remediation is implemented across the Python service, PostgreSQL schema, React application, and Go CLI. Weedout now derives conservative Node/JavaScript reachability from a bounded source inventory, stores evidence, returns it through both API families, renders it in the web application, and prints it in the CLI. Suppressed findings use a distinct `filtered` lifecycle state and are excluded from active work everywhere.

The normal product is Free-only. Paid entitlements, upgrade prompts, user billing routes, and obsolete feature gates were removed. Historical provider data and the legacy `tier='pro'` database value remain readable for rollback/data compatibility, but they do not grant product capabilities or appear as a user-facing plan.

The actual CLI was changed and tested in its own repository, not mocked in the web repository. Feature commit `7f940fc` and Linux credential-directory hardening commit `a7856a5` are on `itsmangoo/weedout-cli` main. The `v0.3.1` cross-platform release workflow completed successfully and published the checksummed assets. The Unix installer was already live; a previously missing `/install.ps1` route is now implemented and is shipped by the web application from the canonical CLI installer.

StrictSeal issue-to-fix mapping

StrictSeal issue	Implemented fix	Verification
Reachability was inferred from dependency depth or severity	Added a source-derived Node analyser with <code>reachable</code> , <code>potentially_reachable</code> , <code>not_observed</code> , and <code>unknown</code> ; severity and relationship depth remain separate fields	StrictSeal unit fixture, API/storage end-to-end test, CLI multipart and packaged-binary tests
Reachability lacked inspectable evidence	Evidence stores source file, line, import kind, imported package, dependency path, and explanation	<code>src/api.js:1 imports axios</code> is asserted through storage/API and printed by the CLI
Analysis uncertainty could look safe	Incomplete collection, unsupported files, unsafe paths, decoding failures, non-literal dynamic imports, and unmappable transitive paths prevent a negative conclusion and yield <code>unknown</code>	Direct unit cases plus malformed/incomplete API cases
Noise/suppressed findings still appeared <code>OPEN</code>	Added <code>AlertStatus.FILTERED</code> , migrated existing suppressed/open rows, and made scan carry-forward preserve dismissal separately	API open/filtered views, dashboard totals, alert rows, email selection, and CLI output tests
CLI docs described commands not in the distributed binary	Aligned real implementations/help/validation for <code>auth</code> , <code>create</code> , <code>findings</code> , <code>rules</code> , <code>scan</code> , <code>init</code> , and <code>version</code> ; removed plan announcement/gating code	Go suite and opt-in test against the built executable, including success/failure paths
Windows installer was advertised at a missing URL	Added public <code>/install.ps1</code> , dual-installer syncing, route inventory and drift tests	PowerShell installer downloaded <code>v0.2.0</code> , verified SHA-256, installed, and ran <code>weedout version</code> ; web route tests pass
Pro remained visible and affected capability gates	Normalized all accounts to one Free product contract, removed paid gates/UI/CTAs and the normal billing route	Backend tier/gate suites and React navigation/settings/pricing tests
Landing claims exceeded the real product	Replaced stale/fabricated modules and screenshots with actual dependency, evidence, filtering, OSV/CISA, CLI, and CI behavior; future modules are visibly <code>Planned</code>	Landing/backend contract tests and production React build
Admin used a separate top navigation	Rebuilt it as a responsive Weedout sidebar with only <code>Overview</code> , <code>Users</code> , <code>Billing history</code> , <code>Inbox</code> , <code>Compose</code> , <code>Docs</code> , and <code>Audit log</code>	13 navigation tests cover real sections, disclosure, Escape, theme, and Back to app

Automated reachability implementation

Inputs and collection

The Go CLI walks the repository deterministically for `.js`, `.jsx`, `.mjs`, `.cjs`, `.ts`, `.tsx`, `.mts`, and `.cts` files. Collection is bounded and excludes dependency/build/VCS directories and symlinks. It sends source files and repository-relative labels in the same multipart scan request as the manifest and optional policy.

The API validates the inventory before analysis:

- maximum 512 source files;
- maximum 512 KiB per file;
- maximum 4 MiB total;
- maximum 100 KB source-context JSON;
- UTF-8 decoding;
- one unique string label per upload;
- no absolute, Windows-drive, traversal, control-character, or unsupported-extension labels.

Source is analysed in memory and discarded. Only the bounded analysis summary and evidence are persisted.

State rules

State	Conservative rule
reachable	A literal import/require of the package was observed in production source
potentially_reachable	The observation is in test/config/non-production source, or an imported direct package provides an inspectable lockfile path to a transitive package
not_observed	Collection is declared complete, static analysis found no uncertainty, and no import/dependency-path observation exists
unknown	Source is absent/incomplete, a non-literal import is present, a file cannot be analysed, or a transitive route cannot be mapped reliably

Dependency relationship (`direct/transitive`), severity, and automated reachability are separate values throughout the model. Manual dismissal reasons remain part of the finding lifecycle and are not used as reachability evidence.

Persistence and presentation

Migration `c4a9b8d7e6f5` adds per-target analysis time/completeness/source-count/notes and per-dependency/per-match state plus JSONB evidence. Scanner results copy the analysed state into finding snapshots so historical finding output does not silently change when a later scan changes the tree.

The internal web API, public CLI API, dashboard coverage panel, project view, finding details, finding rows, and CLI scan/findings output use the same state names and evidence structure.

Known limitations

This is intentionally a conservative static import observer, not a whole-program JavaScript analyser. It does not prove that an imported function is executed, model runtime control flow, resolve bundler aliases, execute plugins, or trace values across calls. Literal ESM imports/exports, `require/require.resolve`, and literal dynamic imports are supported. A non-literal import makes otherwise negative results unknown.

A transitive result is `potentially_reachable`, not `reachable`, because importing the top-level package and observing a lockfile path does not prove the vulnerable code path executes. Non-npm ecosystems retain unknown; no source reachability claim is made for them.

Noise and finding status

Suppression and lifecycle are now aligned:

- triage verdict suppressed is stored as status `filtered`, never open;
- active counts and attention excerpts select actionable open findings only;
- filtered, dismissed, and resolved are independent views;
- dismissing an actionable item does not mutate automated reachability;
- rescans preserve a manual dismissal but refresh dependency/evidence fields;
- alert delivery selects only genuinely new actionable findings;
- migration converts existing `verdict='suppressed' AND status='open'` rows to `filtered`.

CLI and distribution

The CLI now collects bounded Node source context, submits it with scans, parses reachability/evidence, and prints an analysis summary. Plan announcement and Pro gating code were deleted. The API plan model exposes a single Free contract.

Every documented command has real argument parsing and `--help` exits 0. Extra positional arguments fail with exit 2. The CI exit contract remains:

- 0: command ran and nothing blocks (or a non-CI scan reports findings);
- 1: `scan --ci` ran and found a blocking result;
- 2: command did not run because of invalid input, missing configuration, rejected credentials, service failure, or missing manifest.

`weedout init` reads a key from `WEEDOUT_API_KEY` (recommended) or an explicit `--api-key`; it does not prompt for or echo the secret. Packaged tests assert the sentinel credential never appears in `stdout/stderr`.

Installers

Canonical `install.sh` and `install.ps1` live in the CLI repository. `scripts/sync_cli.py` now mirrors both into the web runtime and tests fail if either copy drifts.

- Unix: executed in a clean `alpine:3.20` container; downloaded `weedout-linux-amd64` from release v0.2.0, verified the published SHA-256, installed, and printed `weedout 0.2.0`.
- Windows: executed with pipeline semantics (`Get-Content -Raw install.ps1 | Invoke-Expression`) into an isolated directory; downloaded `weedout-windows-amd64.exe` v0.2.0, verified SHA-256, installed, and printed `weedout 0.2.0`.
- Web: `/install.sh` and `/install.ps1` are public plain-text, short-cache routes; both have content and canonical-copy tests.
- Latest release: after v0.3.1 publication, both installers resolved latest without a `VERSION` override, verified SHA-256, installed the Windows/Linux amd64 assets, and printed `weedout 0.3.1`.

Before this change, live checks returned HTTP 200 for `/install.sh` and HTTP 404 for `/install.ps1`. The PowerShell endpoint therefore requires the normal Weedout web deployment before the public URL can change to 200.

Free-only migration

`FREE_PLAN` is the only product contract: full tree, four-hour schedule, custom rules/profiles, email/Discord/custom webhooks, CLI access, and 365-day history are available because those implementations and tests exist. Legacy Tier . PRO rows map to the exact same limits and all API serializers return `free`.

Removed normal-user surfaces include Pro pricing/comparison copy, upgrade actions, lock badges, feature gates, checkout configuration, internal billing mutation routes, normal billing pages, and pricing navigation. `/billing` and `/billing/success` remain compatibility redirects to settings so old bookmarks do not become unsafe catch-alls. The Dodo webhook and admin billing reconciliation remain only to preserve historical records and provider cleanup.

Landing page cleanup

The dark Weedout visual direction, rounded surfaces, whitespace, and identity remain. The page now leads with the implemented product: dependency vulnerability analysis, dependency paths, automated reachability evidence, filtering, OSV/CISA context, CLI operation, and CI exits. Fake live signals, fabricated customer/social-proof blocks, obsolete stage animations, and screenshots describing nonexistent product modules were removed.

Source code, Secrets, and CI/config analysis modules remain visible only as disabled Panned items. Current product screenshots use the StrictSeal-style packages and real states.

Admin redesign

The admin application now uses the product sidebar proportions, tokens, spacing, typography, surfaces, radius, and mobile disclosure pattern. It retains a clear Admin tag, theme control, current administrator identity, and Back to app action. The sidebar contains only implemented routes: Overview, Users, historical Billing, Inbox, Compose, Docs, and Audit log.

Paid conversion metrics and tier mutation/filter controls were removed. Compose audiences are the implemented all-users or one-user modes. Historical Billing is explicitly operational/reconciliation data and does not change Free product access.

Focused security review

Area	Review/result
API keys	Existing hashed-at-rest, prefix, scope, project binding, indistinguishable 401 behavior retained; scan/read/manage boundaries re-tested
CLI credentials	init no longer prompts/echoes a key; packaged sentinel test checks stdout/stderr; machine and project credentials remain separate; existing Unix config directories are forced to mode 0700
Auth/admin authorization	Protected route inventory, session states, CSRF, admin dependency, suspension, 2FA, and last-admin constraints passed
Ownership/IDOR	Project, finding, rule/profile, key, and admin-action routes use server-side ownership/role checks; focused route tests passed
XSS/injection	React escapes evidence labels; Markdown raw HTML remains disabled; source is never executed; SQL continues through SQLAlchemy parameters
CSRF	Cookie-session mutations retain CSRF checks; bearer-key APIs do not rely on cookies
Path traversal	CLI walker refuses symlinks/out-of-root results; API rejects traversal, absolute/drive/control paths and duplicate labels
Shell execution	Source analyser uses regex parsing only and never runs uploaded project code; installer arguments are fixed/quoted and downloads are checksummed
Validation/limits	Multipart counts, labels, sizes, context JSON, UTF-8, command arguments, and timeouts are bounded
Sensitive errors/logging	Keys are not returned after creation or printed by CLI flows; evidence paths are normalized before storage/output

No concrete authorization bypass, IDOR, injection path, or credential disclosure was found in the reviewed changes. The path-label and duplicate-inventory checks were strengthened during this review.

Tests and build results

Python/web service

- Full suite: **1725 passed, 14 skipped, 2 xfailed**, one pytest-asyncio deprecation warning; 364.14 seconds.
- Focused authorization/authentication/admin/project/finding/API suite: **309 passed**.
- Focused reachability/API suite after path hardening: **53 passed**.
- Installer/route inventory suite: **40 passed**.

- Ruff lint: all checks passed.
- Ruff format: 200 files formatted.
- `git diff --check`: passed (Git emitted only local LF-to-CRLF conversion notices).

Database migration

- Created a blank disposable PostgreSQL database.
- Upgraded the full Alembic history to `c4a9b8d7e6f5` (head).
- `alembic check`: no new upgrade operations detected.
- Downgraded one revision and upgraded back to head.
- Second `alembic check`: no model/schema drift.

React web application

- Vitest default command after deterministic file scheduling: **20 files, 137 tests passed**.
- Focused navigation suite: **13 passed**, including admin mobile disclosure and all real sections.
- ESLint: passed.
- Vite production build: passed; 2,420 modules transformed in 6.24 seconds.
- Main JS bundle: 483.59 kB (149.89 kB gzip); CSS: 117.67 kB (20.97 kB gzip).

The test runner was changed to serial file execution because repeated default parallel runs produced load-dependent lazy-import and five-second user-event timeouts while the exact suite passed serially. This changes test scheduling only, not product behavior or production timeouts.

Go CLI and packaged executable

- `gofmt -l .`: no output.
- `go vet ./...`: passed.
- `go test ./...`: all packages passed.
- Clean `go lang:1.22-bookworm` Linux container `go test ./...`: all packages passed after enforcing mode 0700 on existing credential directories.
- Built with `-trimpath` and stripped linker flags, version `0.0.0-remediation`.
- Packaged executable tests: `TestPackagedCommands` and `TestPackagedArtifactMatchesRuntimePlatform` passed.
- Packaged commands covered root help/version and help, success, and failure paths for `init/auth/create/scan/findings/rules` against a mock HTTP API.
- Binary: `dist/weedout.exe`, 6,883,840 bytes, SHA-256 `EFDF92D22C371291E83536B97A64FFAE2905C3FADF58E22B53C2DCD9A68C1D76`.
- Actual v0.2.0 Windows and Linux release installers passed with checksum verification as described above.
- GitHub Actions release `v0.3.1`: verify, five platform/architecture build jobs, and release publication all passed. Published assets include raw and gzip Linux/macOS amd64/arm64 binaries, raw and gzip Windows amd64, and `checksums.txt`.
- Actual latest-resolution Windows and clean-Alpine installs selected v0.3.1, verified checksums, and ran `weedout version` successfully.

The first `v0.3.0` workflow attempt exposed an existing Linux permission test failure: an already-created credential directory could remain mode 0755. The release stopped before building. The implementation now explicitly applies mode 0700, the exact Linux suite passed locally, and the fix was released under a new immutable `v0.3.1` tag rather than rewriting the failed tag.

Remaining manual verification and operational notes

1. After the Weedout web deployment completes, verify `https://weedout.dev/install.ps1` returns HTTP 200/plain text and run the exact `public irm ... | iex` command in a disposable Windows environment. Local pipeline semantics and the real release download were already tested.
2. A live rendered desktop/mobile inspection could not be completed because the installed in-app Browser plugin referenced a removed older `browser-service.mjs`. Responsive DOM behavior and breakpoints are covered, but a human visual pass at desktop and phone widths remains advisable after deployment.
3. No live Dodo, SMTP, Discord, or customer webhook credentials were used. Automated signature, authorization, payload, and delivery-selection tests passed; production provider dashboards/logs should be checked during rollout.
4. The migration was rehearsed on a clean disposable PostgreSQL database. Production rollout still requires the normal backup, migration window, and post-deploy metrics/log review.

These are deployment/environment verification items, not hidden product implementations. No untested source-code, secrets, or CI/config analyser is advertised as available.